MINI PROJECT PRESENTATION

WeatherNow

Weather Forecast Website

A simple, responsive React application using live weather data — with no database connection.

Built with HTML • CSS • React • Open-Meteo API

1. AIM

Purpose of the mini project

Aim

To design and develop a simple, professional and responsive weather forecast website that allows users to search for a city and view its current temperature, weather condition, humidity and wind speed.

The application uses React for the user interface and the Open-Meteo public API for weather information. It does not use any database.

Objectives

- Search weather by city
- Display current weather details
- Handle invalid city names
- Work on desktop and mobile
- Run locally in VS Code

2. PROJECT EXPLANATION

How the application works

1. User enters a city name in the search box.

2. React handles the form submission and updates the screen.

3. The app calls the Open-Meteo Geocoding API to find the city's latitude and longitude.

4. It sends those coordinates to the Open-Meteo Forecast API.

5. The returned JSON data is displayed as temperature, condition, humidity and wind speed.

6. If a city is not found or the network fails, a helpful error message is shown.

Frontend

HTML + CSS + React

Weather service

Open-Meteo API (no key)

Storage

No database / no server

PROJECT FLOW

Request and response sequence

User

Enters city

→

React app

Handles search

→

Geocoding API

Returns coordinates

→

Forecast API

Returns weather JSON

→

Website

Displays results

Important

Weather data is retrieved from a public internet API. No MongoDB, Express server, login, or database configuration is needed for this version.

3. SOURCE CODE

Only two project source files

File 1 — index.html

Contains the page structure, responsive CSS design, React and Babel CDN links, and the app.js script reference.

File 2 — app.js

Contains the React component, city search form, API requests, weather-code mapping, state updates and error handling.

No database • No backend • No API key • Internet connection required

The complete code for both files is included in the following slides and is also provided in the companion project folder.

3. SOURCE CODE — index.html (Part 1)

Complete source code — copy exactly into the named file

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>WeatherNow | Weather Forecast</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; font-family: Arial, sans-serif; color: #17243a;
      background: linear-gradient(135deg, #eaf4ff, #f8fbff); min-height: 100vh; }
    header { padding: 22px 8%; display: flex; justify-content: space-between;
      align-items: center; background: #fff; box-shadow: 0 2px 12px #183b6412; }
    .brand { font-size: 23px; font-weight: 800; color: #1769aa; }
    .tag { color: #708198; font-size: 13px; }
    main { max-width: 920px; margin: 42px auto; padding: 0 20px; }
    h1 { margin-bottom: 8px; font-size: 30px; }
    .intro { color: #6b7b91; margin-top: 0; }
    .search { display: flex; gap: 10px; margin: 25px 0; }
    input { flex: 1; padding: 14px 16px; border: 1px solid #d6e1ee;
      border-radius: 10px; font-size: 15px; outline-color: #2584ce; }
    button { border: 0; border-radius: 10px; padding: 0 22px; color: white;
      background: #176fba; font-weight: 700; cursor: pointer; }
    button:hover { background: #105991; }
    .card { background: white; border: 1px solid #e2ebf4; border-radius: 18px;
      padding: 28px; box-shadow: 0 12px 32px #23496b10; }
    .top { display: flex; justify-content: space-between; gap: 15px; align-items: center; }
    .place { color: #61738a; font-size: 16px; }
    .temp { font-size: 56px; font-weight: 800; color: #1769aa; margin: 12px 0 4px; }
    .condition { font-size: 18px; color: #53677f; }
    .weather-icon { font-size: 64px; }
    .stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; margin-top: 25px; }
    .stat { background: #f3f8fd; padding: 16px; border-radius: 12px; }
```

Save as index.html. CSS continues on the next slide.

3. SOURCE CODE — index.html (Part 2)

Complete source code — copy exactly into the named file

```
    .stat small { display: block; color: #7b8da2; margin-bottom: 8px; }
    .stat strong { font-size: 19px; }
    .status { margin: 14px 0; color: #b14c3c; min-height: 18px; }
    footer { text-align: center; color: #8493a6; font-size: 12px; margin: 28px 0; }
    @media (max-width: 600px) {
      header { padding: 18px 5%; } main { margin: 28px auto; }
      .search { flex-direction: column; } button { padding: 14px; }
      .stats { grid-template-columns: 1fr; } .temp { font-size: 46px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">🌤 WeatherNow</div>
    <div class="tag">Simple • Live • Local</div>
  </header>
  <div id="root"></div>
  <script type="text/babel" src="app.js"></script>
</body>
</html>
```

This completes index.html.

3. SOURCE CODE — app.js (Part 1)

Complete source code — copy exactly into the named file

```
const { useState } = React;

function weatherText(code) {
  if (code === 0) return ["Clear sky", "☀️"];
  if ([1, 2, 3].includes(code)) return ["Partly cloudy", "⛅"];
  if ([45, 48].includes(code)) return ["Foggy", "🌫️"];
  if ([51, 53, 55, 61, 63, 65, 80, 81, 82].includes(code)) return ["Rain", "🌧️"];
  if ([71, 73, 75, 77, 85, 86].includes(code)) return ["Snow", "❄️"];
  if ([95, 96, 99].includes(code)) return ["Thunderstorm", "⛈️"];
  return ["Current conditions", "🌤️"];
}

function App() {
  const [city, setCity] = useState("Vijayawada");
  const [query, setQuery] = useState("Vijayawada");
  const [weather, setWeather] = useState(null);
  const [message, setMessage] = useState("Search for a city to view its current weather.");
  const [loading, setLoading] = useState(false);

  async function getWeather(event) {
    event.preventDefault();
    if (!query.trim()) {
      setMessage("Please enter a city name.");
      return;
    }
    setLoading(true);
    setMessage("");
    setWeather(null);
    try {
      const geoUrl = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(query.trim())}&count=1&language=en&format=json`;
      const geoResponse = await fetch(geoUrl);
      if (!geoResponse.ok) throw new Error("Unable to contact the location service.");
      const geoData = await geoResponse.json();
      if (!geoData.results || geoData.results.length === 0) {
        throw new Error("City not found. Check the spelling and try again.");
      }
      const location = geoData.results[0];
      const forecastUrl = `https://api.open-
```

Save all parts in order as one file named app.js.

3. SOURCE CODE — app.js (Part 2)

Complete source code — copy exactly into the named file

```
    setWeather(forecastData.current);
    setMessage("");
  } catch (error) {
    setMessage(error.message || "Something went wrong. Please try again.");
  } finally {
    setLoading(false);
  }
}

const condition = weather ? weatherText(weather.weather_code) : null;

return (
  <main>
    <h1>Weather Forecast</h1>
    <p className="intro">Check current weather conditions for any city.</p>
    <form className="search" onSubmit={getWeather}>
      <input value={query} onChange={e => setQuery(e.target.value)}
        placeholder="Enter city name" aria-label="City name" />
      <button type="submit" disabled={loading}>{loading ? "Loading..." : "Search"}</button>
    </form>
    <div className="status" role="status">{message}</div>
    {weather && (
      <section className="card">
        <div className="top">
          <div>
            <div className="place">Current weather in</div>
            <h2>{city}</h2>
            <div className="temp">{Math.round(weather.temperature_2m)}°C</div>
            <div className="condition">{condition[0]}</div>
          </div>
          <div className="weather-icon" aria-hidden="true">{condition[1]}</div>
        </div>
        <div className="stats">
          <div className="stat"><small>Humidity</small><strong>{weather.relative_humidity_2m}%</strong></div>
          <div className="stat"><small>Wind speed</small><strong>{weather.wind_speed_10m} km/h</strong></div>
          <div class="stat"><small>Temperature</small><strong>{weather.temperature_2m}°C</strong></div>
        </div>
      </section>
```

Save all parts in order as one file named app.js.

3. SOURCE CODE — app.js (Part 3)

Complete source code — copy exactly into the named file

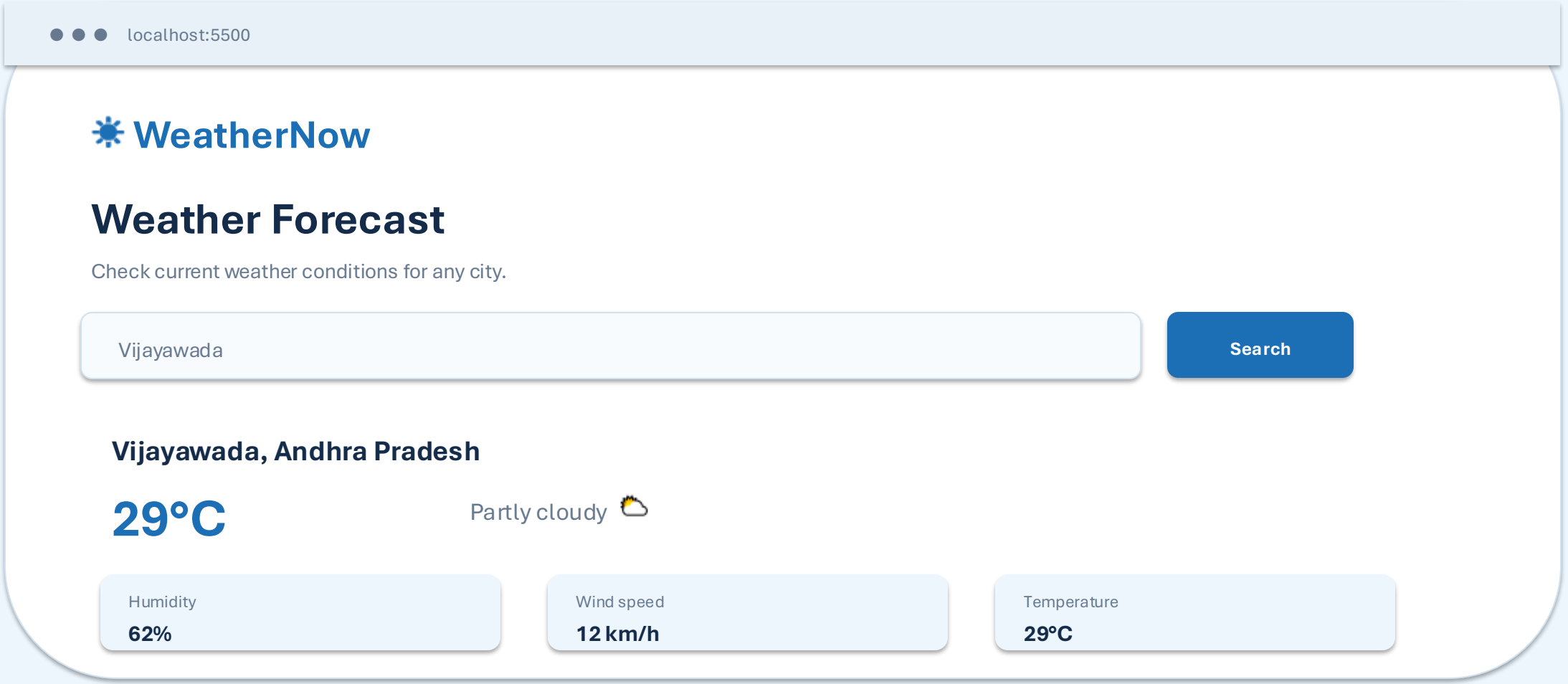
```
}

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

Save all parts in order as one file named app.js.

4. EXPECTED OUTPUT

What the user should see after searching for a city



5. OUTPUT

Sample display (illustrative layout)

Successful search

City: Vijayawada

Temperature: 29°C

Condition: Partly cloudy

Humidity: 62%

Wind speed: 12 km/h

Other possible outputs

- City not found: a clear error message appears.
- Empty search: the app asks the user to enter a city.
- Network issue: the app displays an error message.

Note: actual values change according to the live API response.

The values above are examples only; real weather values depend on the selected city and time.

HOW TO RUN IN VS CODE

Setup steps

1. Install Visual Studio Code and open the weather_forecast_project folder.
2. Make sure index.html and app.js are in the same folder.
3. Install the Live Server extension in VS Code.
4. Right-click index.html and select Open with Live Server.
5. Wait for the page to open in your browser, enter a city and click Search.
6. Keep an internet connection active so the API can return current weather data.

Requirements

Browser + Internet + VS Code

6. RESULT

The WeatherNow website was designed and developed successfully.

It allows users to search for a city and view current weather information through a simple and responsive interface. The project runs in VS Code using only two source files and does not require any database connection.

Thank You